Algoritmi paraleli si distribuiți

11. Algoritmi distribuiți: Alegerea liderului, Map-Reduce

+ Hadoop

Alegerea liderului

- Context
  - Avem un sitem distribuit, de exemplu procese într-o rețea
- Scop
  - Procesele trebuie să aleagă un lider, adică un proces *principal*
  - Procesul lider ales:
    - Va fi organizatorul unui task distribuit, de exemplu
      - Rădăcina unui APM pentru rețea
      - Inițiatorul unui algoritm de reconstruire a token-ului pierdut intr-o rețea inel.
- Constrângeri referitoare la procesul de alegere
  - Se pornește de la situația în care procesele nu au nici o intuiție referitoare la procesul care va deveni lider
  - Siguranță (en., safety): Doar un singur proces este ales ca lider
  - Liveness: în cel mai rău caz trebuie ales un lider

Alegerea liderului (cont.)

- Algoritmul de alegere trebuie să fie descentralizat
  - Inițiatorii pot fi mulțimi nevide de procese
  - Toate procesele trebuie să execute același algoritm
    - Se preîntâmpină astfel situația în care un proces decide unilateral ca ”Eu sunt liderul!”
  - Procesele au ID-uri distincte și formază o mulțime total ordonată.
    - Unicitatea ID-ului este esențială pentru ca algoritmul de alegere sa te termine întotdeauna.
- Algoritmii sunt exemplificați in rețele de tip inel orientat
  - Procesele stiu ce este in sânga (i.e., in sesul acelor ce ceas) și în dreapta (i.e., in sesul invers acelor de ceas.)

Rețea inel

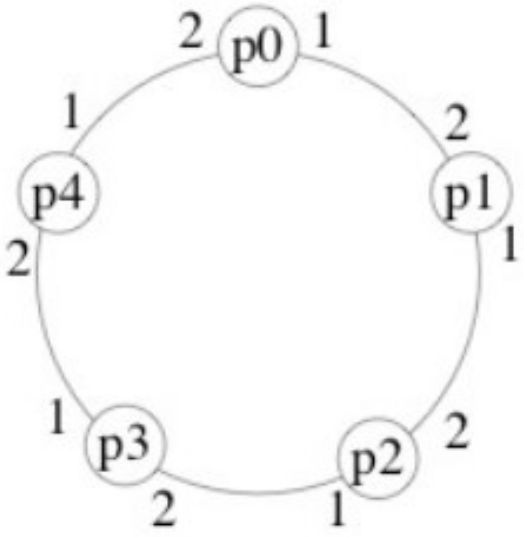
1 = stânga = in sesul acelor ce ceas

2 = dreapta = in sesul invers acelor ce ceas

De exemplu, dacă mesajele sunt transmise pe canalul 1 atunci ele vor traversa rețeaua inel în sensul acelor de ceas.

Motivația pentru studiul rețelei inel:

- Este ușor de analizat
- Rezultatele se aplică pentru topologii oarecare/arbitrare



Alegerea liderului: Definiție

- Fiecare procesor poate avea două stări: ELECTED (ro., ales) sau NOT-ELECTED (ro., neales).
- În momentul în care procesorul trece în starea ELECTED el va rămâne pentru totdeauna în starea ELECTED
  - La fel se întâmplă și pentru trecerea în starea NOT-ELECTED.
  - Obs: Acesta este principiul ireversibilibății deciziei luate.
- În timpul execuției
  - Orice procesor poate trece în starea ELECTED sau NOT-ELECTED
- La finalul execuției
  - Doau un singur procesor (i.e., **liderul**) se va afla în starea ELECTED.

Alegerea liderului: Algoritmi

- Algoritmii pentru alegerea liderului trebuie să aibă în vedere următoarele caracteristici ale rețelei
  - Inelul este anonim/ne-anonim
    - Anonim = procesoarele nu au id-uri unice!
  - N (i.e., dimensiunea rețelei) este o valoare
    - Cunoscută: inel/rețea ne-uniform(ă)
    - Necunoscută: inel/rețea uniform(ă)
  - Algoritmul este sincron/asincron
- Inel sincron anonim
  - Fiecrae procesor execută același algoritm

Algoritmi uniformi în rețele anonime

- Algoritmii uniformi nu iau in calcul dimensiunea rețelei (i.e., avem același algoritm indiferent de dimensiunea rețelei)
  - Toate procesoarele - indiferent de dimensiunea rețelei – sunt modelate cu ajutorul aceleiași mașini de stări.
- Algoritmii ne-uniformi iau în calcul dimensiunea rețelei
  - Algoritmi diferiți pentru fiecare dimensiune a rețelei
  - Obs: Pentru fiecare valoare N, procesoarele din rețea vor fi modelate cu aceeași mașină de stări A_N
- Obs: Aceste rețele sunt anonime, adică nu au ID-uri unice.

Alegerea liderului în rețelele anonime

Teoremă: Nu există nici un algoritm de alegere a liderului pentru rețelele anonime, chiar dacă se cunoaște dimensiunea rețelei (i.e., este neuniformă) și rețeaua este sincronă.

Demonstrație (schiță):

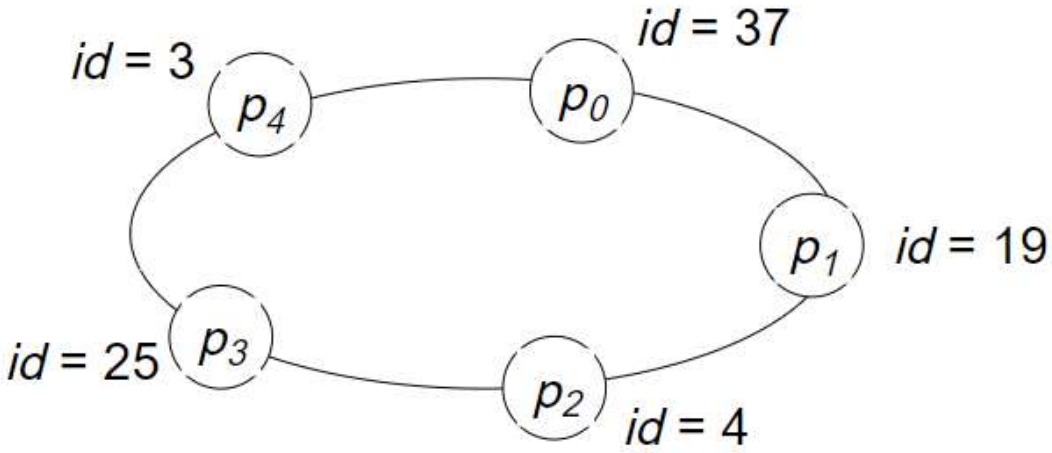
- Toate procesoarele încep dintr-o anumită stare și deci transmit același mesaj (atenție, rețeaua este anonimă)
- Toate procesoarele primesc același mesaj, efectuează aceeași tranziție și transmit același mesaj, de fiecare data!
- La un moment dat, dacă un procesor ajunge in starea ELECTED atunci și toate celelalte procesoare vor ajunge tot in starea ELECTED.
  - Astfel, nu sunt îndeplinite criteriile de siguranță și liveness.
  - Obs: Rezultatul este valabil și în rețelele uniforme (i.e., cu N cunoscut) și asincrone.

Rețele inel cu identificatori (i.e., neanonime)

- Fiecare procesor are un ID unic.
  - NU trebuie confundate pozițiile/indecșii cu ID-urile.
    - Pozițiile/indecșii sunt de la 0 la N-1, sunt folosiți doar pentru analiză și nu sunt disponibili procesoarelor
    - ID-urile sunt valori intregi nenegative care sunt cunoscute procesoarelor prin variabila locală ***id***.
- Definirea unei rețea inel

Parcurgeți rețeaua alăturată de la cel mai mic id către stânga (ordinea acelor de ceas)

3, 37, 19, 4, 25



Algoritmul Chang-Roberts: abordare [rețea inel neanonimă asincronă]

Fiecare proces execută următoarea logică:

- Transmite un mesaj către stânga (în sensul acelor de ceasornic) cu ID-ul propriu, dacă nu a primit un mesaj cu un ID mai mare.
- Transmite către stânga orice mesaj cu ID mai mare decât ID-ul propriu.
- Dacă procesul primește ca mesaj ID-ul propriu atunci el este liderul.
- Obs: Numărul de procesoare nu trebuie să fie cunoscut de algoritm.

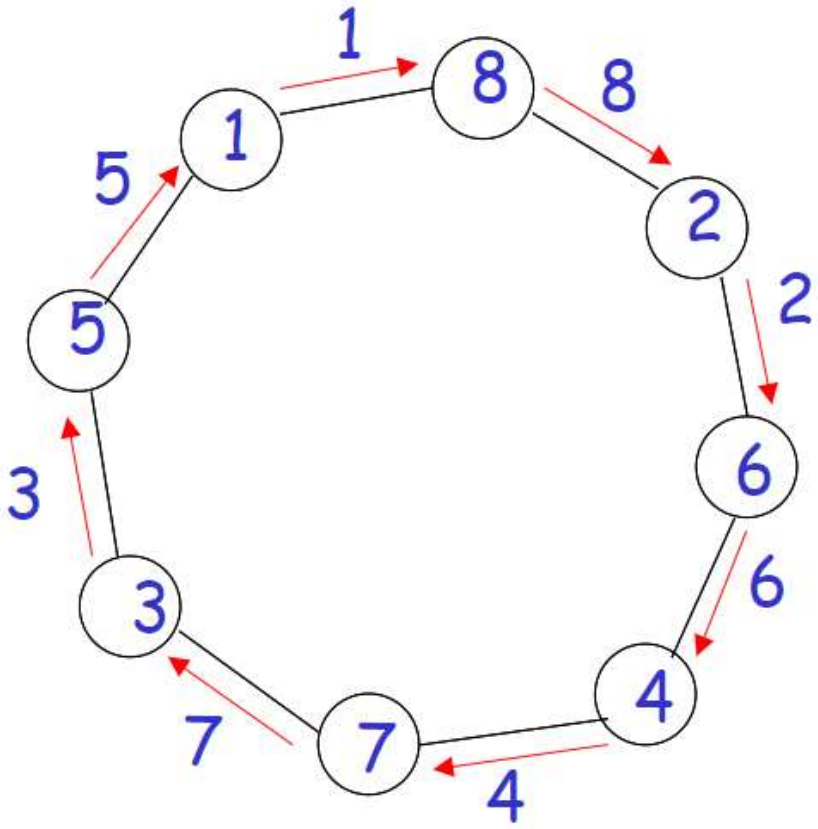
Algoritmul Chang-Roberts [pseudocod pentru procesul Pi]

```
state := cadidat;
send(my_ID);
while (state != leader)
        do receive (vecin_ID)
                if (vecin_ID > my_ID) then send (vecin_ID);
        #end do
        if (vecin_ID = my_ID) then state := leader.
#end_while
```

Algoritmul Chang-Roberts [exemplu de execuție]

Fiecare nod trimite un mesaj către stânga.

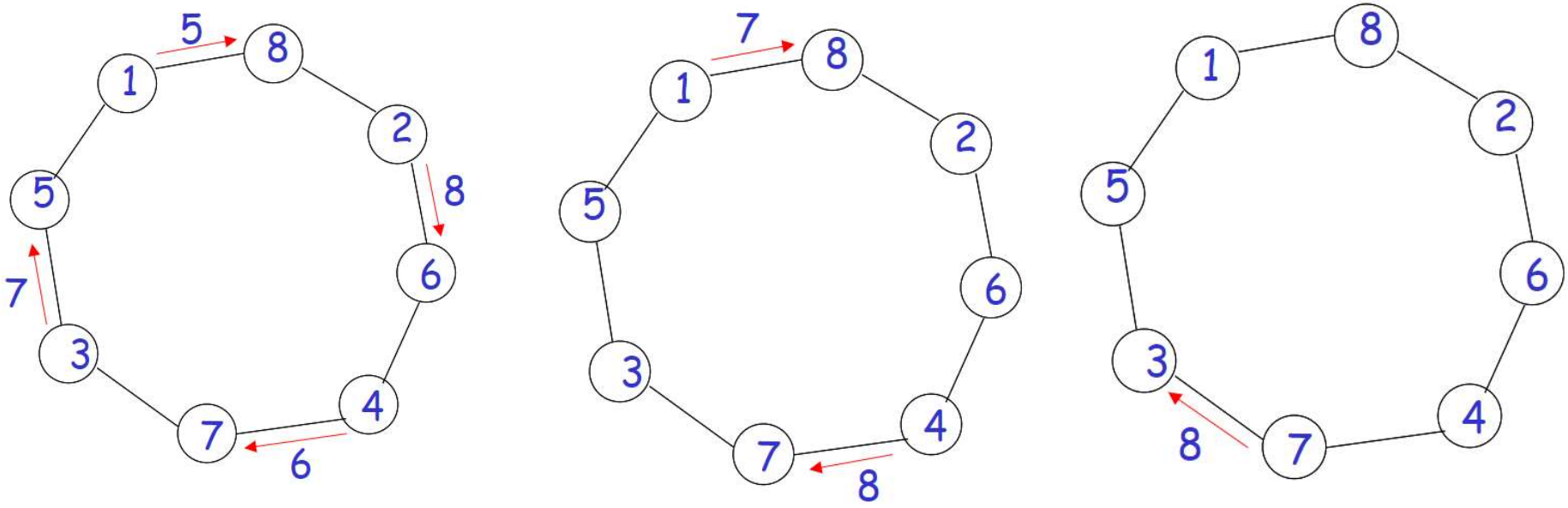
Mesajul conține ID-ul nodului care trimite.



Algoritmul Chang-Roberts
[exemplu de execuție] (cont.)

Dacă: ID-ul mesajului primit > ID-ul nodului current

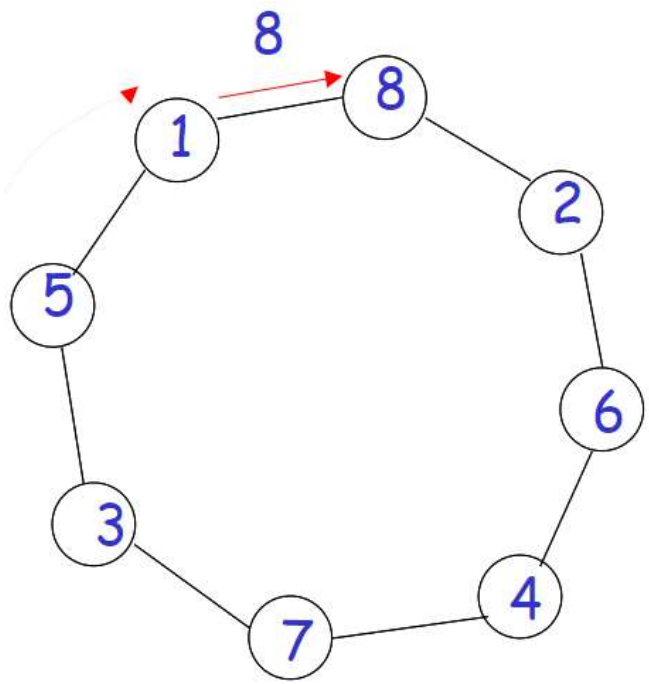
Atunci: Transmite ID-ul primit mai departe

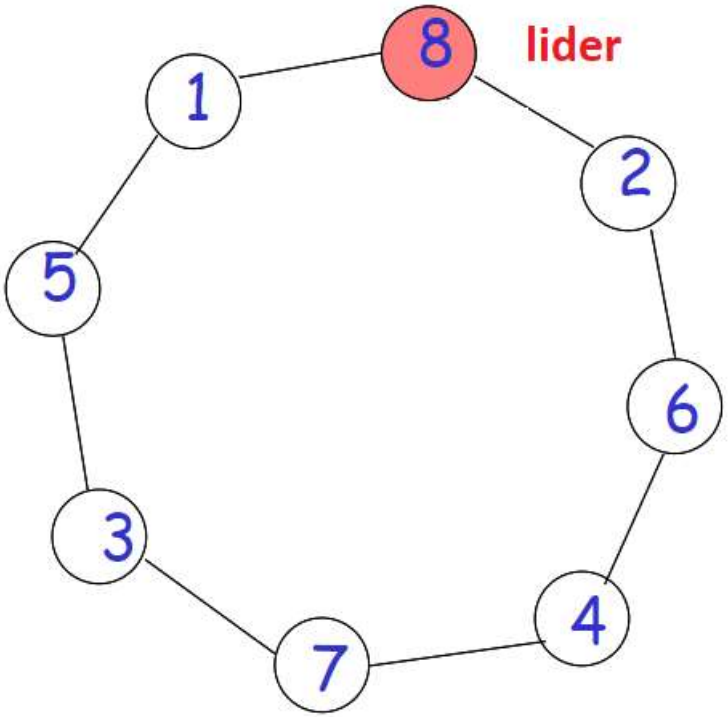


Algoritmul Chang-Roberts [exemplu de execuție] (cont.)

Dacă: Un nod primește propriul mesaj

Atunci: Nodul devine lider





Algoritmul Chang-Roberts: Analiză

- Corectitudinea:
  - Algoritmul alege procesorul cu cel mai mare ID.
  - Mesajul care conține cel mai mare ID trece pe la toate procesoarele.
- Complexitatea comunicației depinde de modul în care sunt distribuite ID-urile pe procesoare
  - ID-ul cel mai mare parcurge intreg inelul: sunt transmise n mesaje
  - Al 2-lea cel mai mare ID parcurge inelul până când ajunge la procesorul cu ID-ul cel mai mare, etc...
  - Cazul cel mai defavorabil costă $O(n^2)$, atunci când procesoarele sunt on ordine descrescătoare: $1+2+...+n = O(n^2)$
  - Cazul cel mai favorabil costă $O(n)$: $n+1+1+...+1 = O(n)$
  - În medie, costul este $O(n \log n)$, în cazul în care toate ID-urile sunt echiprobabile.

Algoritmul lui Franklin: inel nedirecționat asincron

- Scop: Alegerea liderului, adică a nodului cu ID-ul maxim.

Context

- Inelul este bidirecțional

Abordare generală

- Alegerile sunt efectuate in faze (asincron), pe mulțimi din ce in ce mai mari
- Pi este lider în faza ***r*** = 0,1, ...,

<==>

Pi are cel mai mare ID dintre toate nodurile care se află la o distanță de maxim $\boldsymbol{2^r}$

- Faza ***r*** necesită cel mult $\mathbf{4*2^r}$ pași

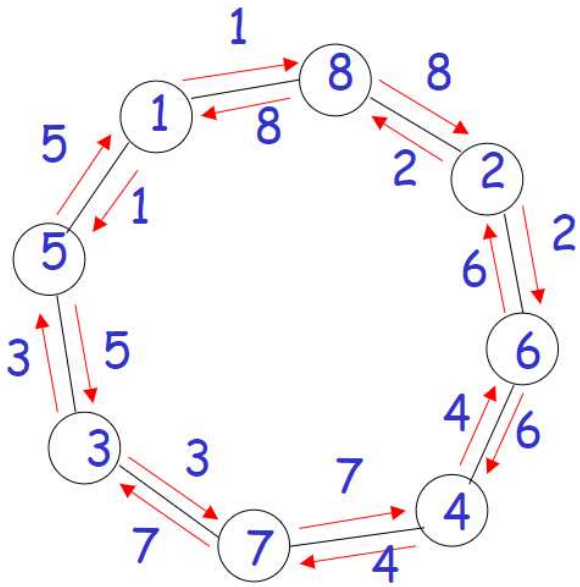
Algoritmul lui Franklin (cont.)

- K-vecinătate
  - Sunt nodurile care se află la distanța de maxim k de nodul selectat.
- Abordare
  - Numai procesele care câștigă alegerile în faza **r** continuă pentru faza **r+1**
  - Dacă un proces primește înapoi mesaj cu propriul ID atunci acesta devine LIDER.
  - Obs: Algoritmul este uniform, adică numărul de procese nu trebuie cunoscut dinainte.

Algoritmul lui Franklin: exemplu de execuție

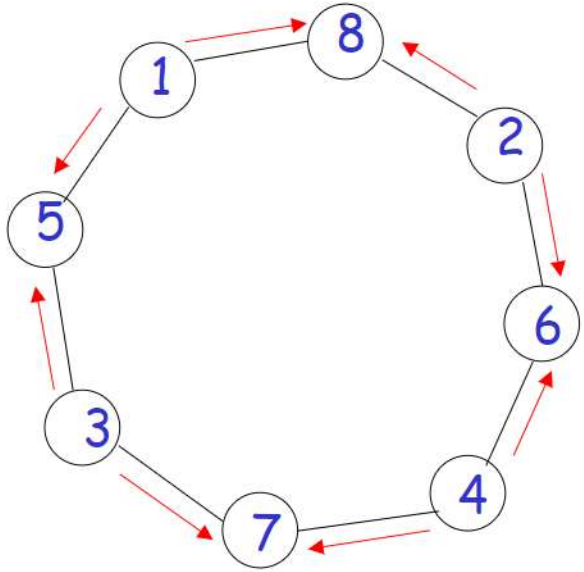
Faza 0:

send(ID, faza, pasul) către 1-vecinătate



If: ID_primit > ID_curent

Then: Trimite răspuns OK



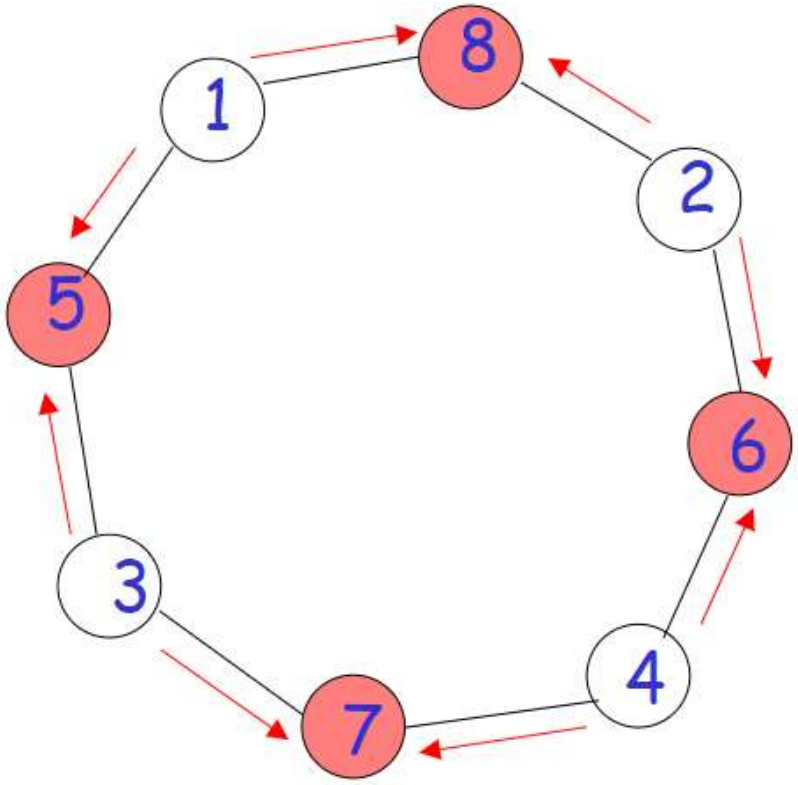
Algoritmul lui Franklin: exemplu de execuție (cont.)

If: un nod primește ambele răspunsuri

Then: nodul devine lider temporar

si trecem la faza următoare

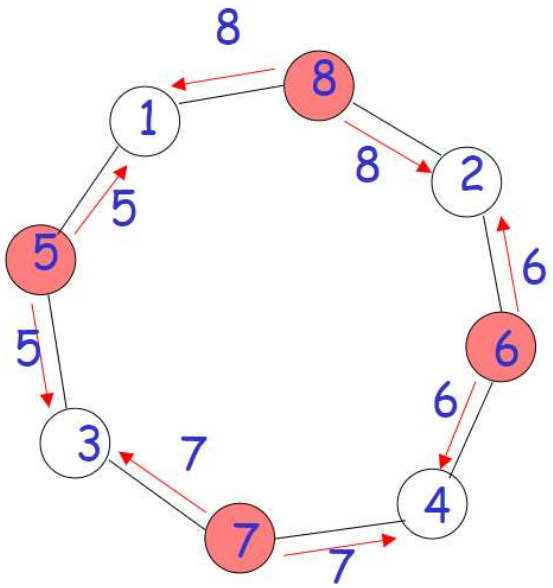
În urma fazei 0 avem următorii lideri temporari: 5, 6, 7 și 8.



Algoritmul lui Franklin: exemplu de execuție (cont)

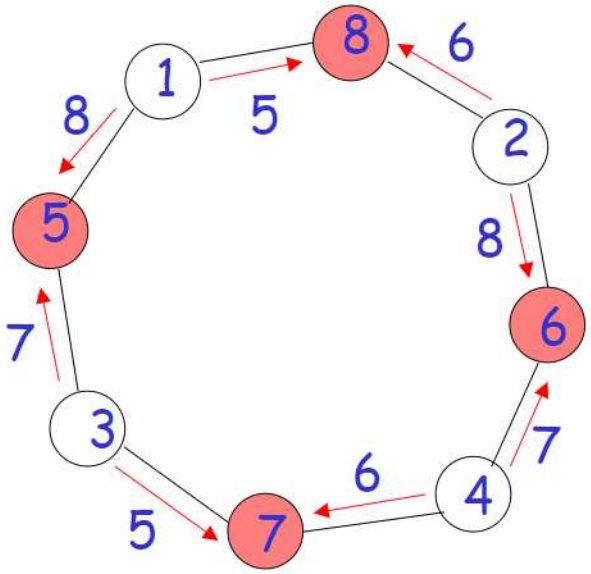
Faza 1: // send(ID, faza, pasul)

send(ID, 1, 1) către 2-vecinătate



If: ID_primit > ID_curent

Then: forward (ID, 1, 2)

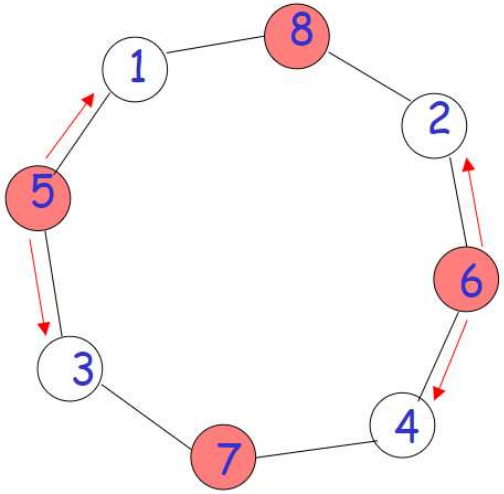


Algoritmul lui Franklin: exemplu de execuție (cont)

La pasul 2 am ajus deja la granița 2-vecinătății

If: ID_primit > ID_curent

Then: trimite răspuns (ID_primit)

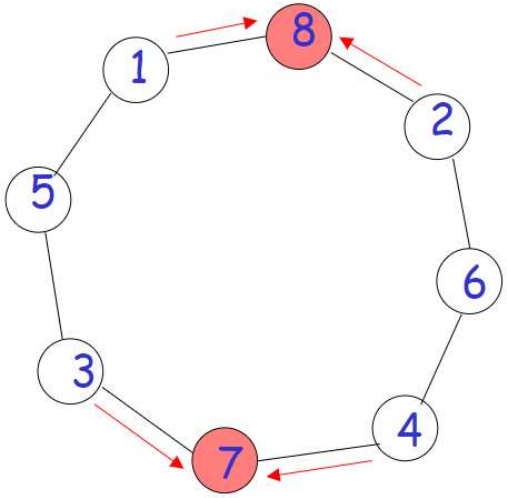


If: ID_primit != ID_curent

Then: forward (ID_primit)

If: un nod primește ambele răspunsuri

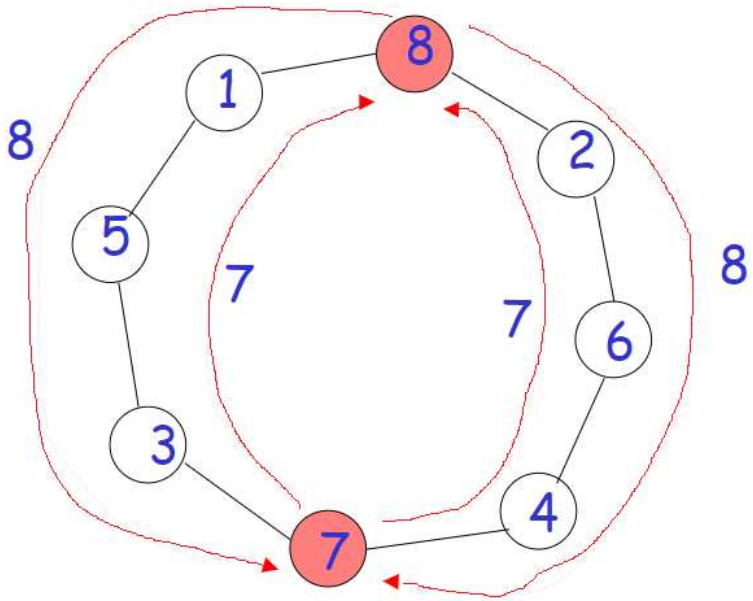
Then: nodul devine lider temporar



Algoritmul lui Franklin: exemplu de execuție (cont)

Faza 2: // send(ID, faza, pasul)

send(ID, 2, 1) către 2^2-vecinătate

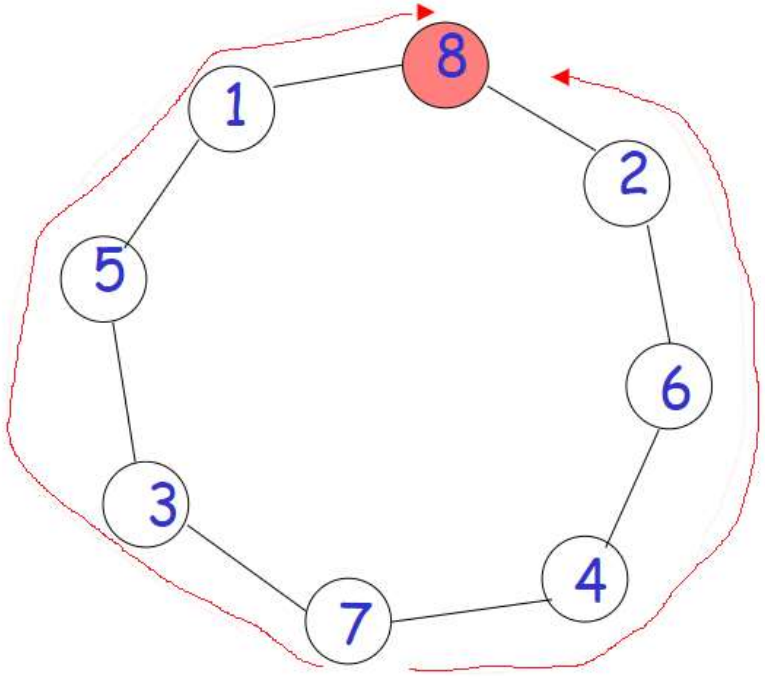


La pasul 2^2: If: ID_primit > ID_current

Then: trimite răspuns (ID_primit)

If: un nod primește ambele răspunsuri

Then: nodul devine lider temporar

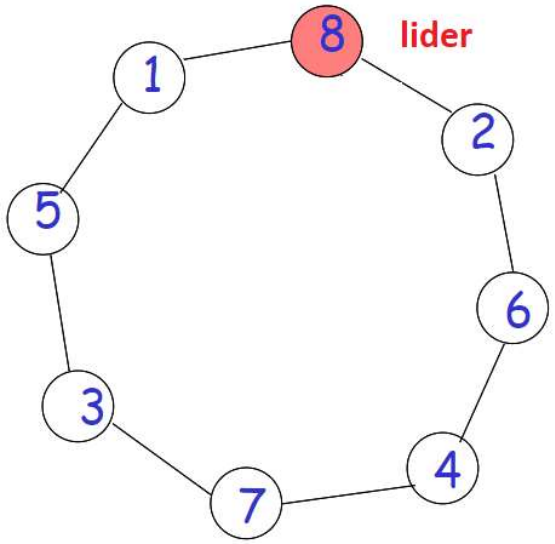


Algoritmul lui Franklin: exemplu de execuție (cont)

Faza 3:

send(ID, 3, 1) către 2^3-vecinătate

Nodul 8 va primi înapoi propriul ID și atunci devine lider.



In general: **N noduri => log n faze**

În faza i: ID-ul se trimite la 2^{i-1} –vecinătate

Complexitatea comunicației:

- Fiecare mesaj aparține unei faze și este inițiat de un anumit proces.
- Nr. de mesaje inițiate de un proces în faza i este cel mult **$4*2^i$** (cu trimiteri si răspunsuri în ambele direcții)

Algoritmul lui Franklin: exemplu de execuție (cont)

Complexitatea comunicației:

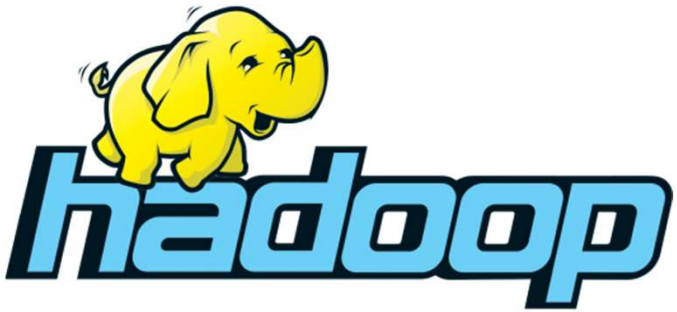
Nr. max. de mesaje / lider		Nr. max. de lideri curenți	
Faza 1: 4	x	n	=4n
Faza 2: 8	x	n/2	=4n
...			
Faza i: 2^{i+1}	x	$n/2^{i-1}$	=4n
...			
Faza log n: $2^{\log n +1}$	x	$n/2^{\log n-1}$	=4n

Nr. total de mesaje = O(n logn)

Algoritmul lui Franklin: concluzie

- Algoritmul O(n logn) este mai complicat decât cel $O(n^2)$ dar folosește mai puține mesaje în cazul cel mai defavorabil.
- Funcționează pentru inel sincron si asincron
- Numărul de mesaje nu poate fi redus în rețelele asincrone.
- Pentru rețea sincronă și neuniformă (i.e., n este cunoscut) un algoritm eficient rezolvă problema în O(n).

Hadoop



Referințe

- Venner, J. (2009). Pro hadoop. Apress.
- White, T. (2012). Hadoop: The definitive guide. " O'Reilly Media, Inc.".
- https://hadoop.apache.org/

Despre Apache Hadoop

- Ce este Apache Hadoop?
  - Este un framework software open source (sub licență Apache) proiectat pentru stocarea și procesarea de date mari pe clustere de hardware comun.
- Cine a creat Hadoop?
  - Doug Cutting și Mike Carafella în 2005.
- Când se folosește Hadoop?
  - La procesarea de texte mari
  - La asamblarea genomului
  - La procesarea grafurilor
  - În Machine learning
  - La analizarea rețelelor sociale mari

Cine folosește Hadoop?

Ecosistemul Hadoop

- Hadoop Common:
  - Conține bibliotecile principale și alte module
- HDFS
  - Hadoop Distributed File System: *a self-healing high-bandwidth clustered storage*
- Hadoop MapReduce:
  - Un model de programare pentru procesarea de date mari

Folosiți unealta potrivită pentru task-ul avut

Baze de date relaționale	Vs	Hadoop
Se folosesc atunci când:		
- Procesarea interactivă OLAP* (<1sec)		- Avem sau nu date structurate (flexibilitate)
- Avem tranzacții ACID		- Scalabilitate a procesărilor și a dimensiunii datelor procesare
- Interogările sunt 100% SQL conforme		- Procesarea datelor complexe

OLAP = Online analytical processing

Stocarea datelor

- Avem două tipuri de noduri: de date și de procesare
- Abordarea:
  - La rulare, datele sunt copiate în nodurile de procesare: acest mod de lucru este viabil numao când lucrăm cu cantități relativ mici de date.

- Ce înseamnă mult?
  - Facebook (500 TB pe zi); Yahoo (peste 170 PB); eBay (peste 6 PB)
  - Aducerea de date la procesoare devine un *bottleneck*!

Cerințe pentru Hadoop:

- Trebuie să gestioneze defecțiunile (en., failure) unor componente astfel încât să nu fie pierdute date
  - Rezultatul final nu trebuie sa fie afectat

- Trebuie să fie scalabil:
  - Creșterea cantității de date trebuie să aibă ca efect scăderea uniformă a performanței pentru toate task-urile
    - NU întreruperea sistemului
  - Creșterea de resurse alocate trebuie să aibă ca efect creșterea capacității de procesare.

Hadoop

- Dean, J., & Ghemawat, S. (2004). MapReduce: Simplified data processing on large clusters.
- Abordare:
  - Se distribuie datele pe mediul de stocare [în HDFS].
  - Fiecare nod poate apoi executa calculele necesare pe datele pe care le deține fără a fi nevoie de mutarea acestora pentru procesarea inițială.

Concepte de bază

- Aplicațiile sunt dezvoltate într-un limbaj de programare de nivel înalt (i.e., Java)
  - NU este nevoie de programare în rețea sau administrarea dependințelor temporale sau a concurenței
- Nodurile trebuie să comunice cât mai puțin între ele
  - Arhitectura este de tip ”nimic partajat” (en., shared nothing)
- Datele sunt distribuite între sisteme de la bun început (cu redundanță)
  - Se efectuează procesările acolo unde datele sunt deja existente

Fluxul de procesare

- Se încarcă datele în sistem prin împărțirea acestora în blocuri
  - De obicei, dimensiunea unui bloc este 64MB sau 128MB;// se folosește redundanța împotriva defectelor!
- Task-urile se împart în două faze:
  - **Map**: care se execută pe blocuri mici de date, acolo unde se află acestea
  - **Reduce:** care combină datele pentru a produce rezultatul final
- Un **program master** care alocă task-urile pe noduri
  - Detectează defectele și reasignează task-urile altor noduri
  - Repornește un task fără a afecta nodurile care procesează alte blocuri de date
  - Adaugă nodurile repornite la sistem și alocă task-urile noi

HDFS (Hadoop Distributed File System)

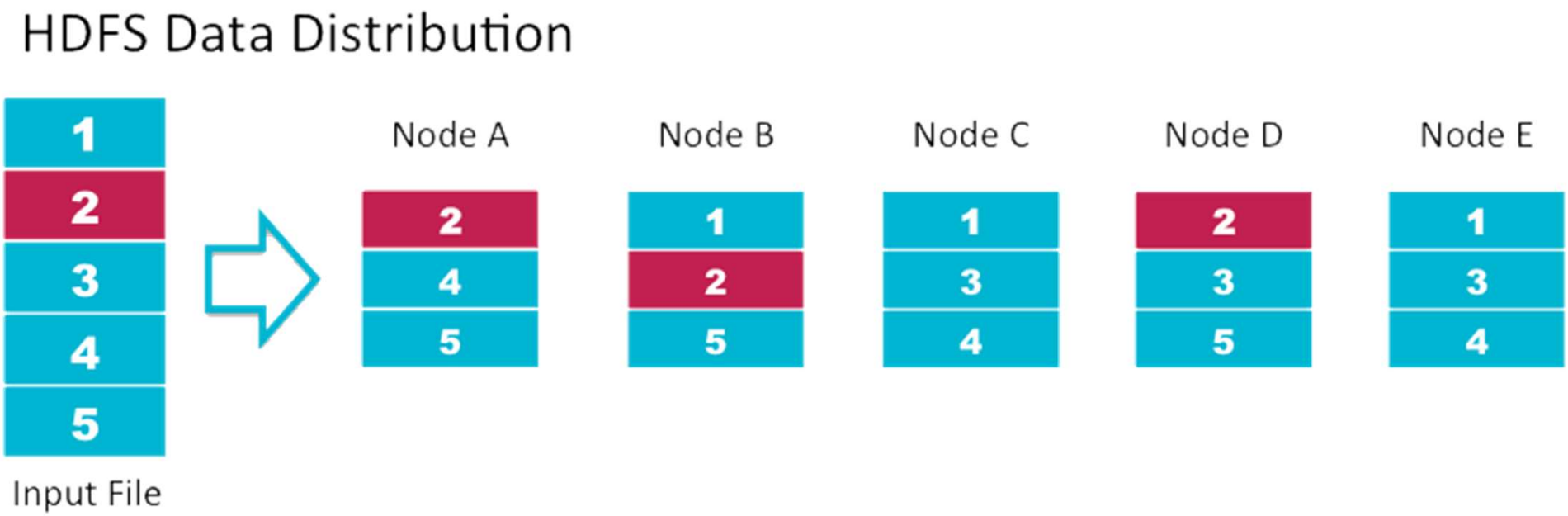
- Gestionează datele din cluster
- Împarte datele în blocuri și le distribuie la noduri
- Crează replici ale blocurilor de date
- Este implementat în Java și are la baza GFS de la Google
- Asigură capacități de stocare redundante pentru cantități mari de date
- Funcționează cel mai bine cu un număr mic de fișiere mari
  - Millioane și nu miliarde de fișiere
  - De obicei, 100MB sau mai mult pentru un fișier
- Este optimizat pentru fluxuri de citire a fisierelor mari

Stocarea fișierelor

- Fișierele se împart în blocuri (en., blocks/chunks)
- La încărcarea fișierului în HDFS blocurile sunt distribuite/salvate pe mai multe sisteme/mașini
  - Se salvează blocuri diferite (ale aceluiași fișier) pe mai multe sisteme diferite
- Blocurile sunt duplicate/salvate pe mai multe mașini
- ***NameNode*** getionează modul în care a fost împărțit un fișier și mașinile unde au fost salvate blocurile acestuia.
  - Acest mod de lucru asigură caracterul distribuit al gestionării fișierelor și asigură disponibilitatea datelor (i.e., a fisierelor) chiar dacă un sistem se defectează.

Replicarea datelor

- Replicarea 3-fold [implicită]



Scenariu implicit/de bază

- Fișierul este împărțit în blocuri (dimensiunea implicită=64MB), iar blocurile sunt replicate în cluster (implicit de 3 ori, fiecare bloc mai are două copii).
- Acest scenariu este optimizat pentru:
  - Rată de transfer (en., throughput) cât mai mare
  - Operații *Put*/*Get*/*Delete*
  - Operații *Appends*
- Replicarea blocurilor asigură:
  - Durabilitatea
  - Disponibilitatea
  - Rata de transfer

Regăsirea datelor

- Atunci când un (cod) client dorește să regăsească date (i.e., un fișier)
  - Trebuie să comunice cu ***NameNode*** pentru a detrmina care blocuri compun fișierul și pe care noduri se află acestea.
  - Apoi, clientul comunică direct cu nodurile de date pentru a citi datele
  - Citirea se realizează transparent, clientul nu *stie* modul în care a fost distribuit fișierul.

MapReduce: Distribuirea procesării între noduri/sisteme

- MapReduce : Prezentare generală
  - Fiecare nod procesează datele stocate local
  - Cosntă din 2 faze principale
    - Map
    - Reduce

- MapReduce : Caracteristici
  - Paralelizare și distribuție automată
  - Toleranță la erori/defecțiuni (en., Fault-Tolerance)
  - Oferă o mod de lucru abstract pe care programatorii îl pot utiliza cu ușurință

The Mapper

- Datele se citesc ca perechi cheie/valoare
  - La cheie de obicei se renunță
- Răspunsul este sub forma perechilor cheie/valoare

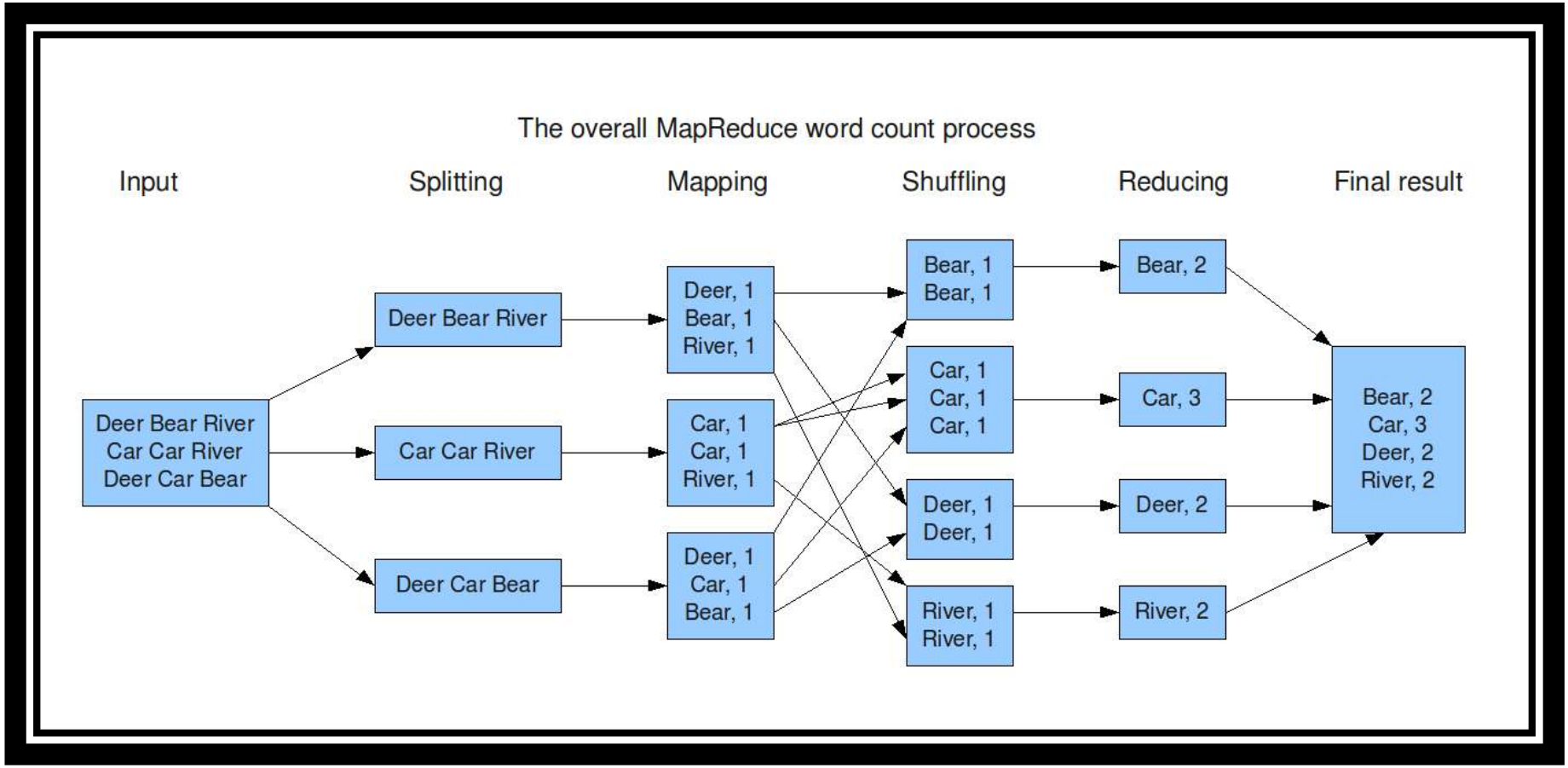
Amestecă (en., shuffle) și **sortează**

- Răspunsul de la mapper este sortat după cheie
- Toate valorile cu aceeași cheie vor merge garantat pe aceeași mașină

The Reducer

- Este apelat o singură dată pentru fiecare cheie unică
- Primește o listă de valori cu cheia ca intrare
- Răspunsul este format din perechi cheie/valoare
  - De obicei avem un singur răspuns pentru fiecare cheie primită ca intrare

Numărarea cuvintelor



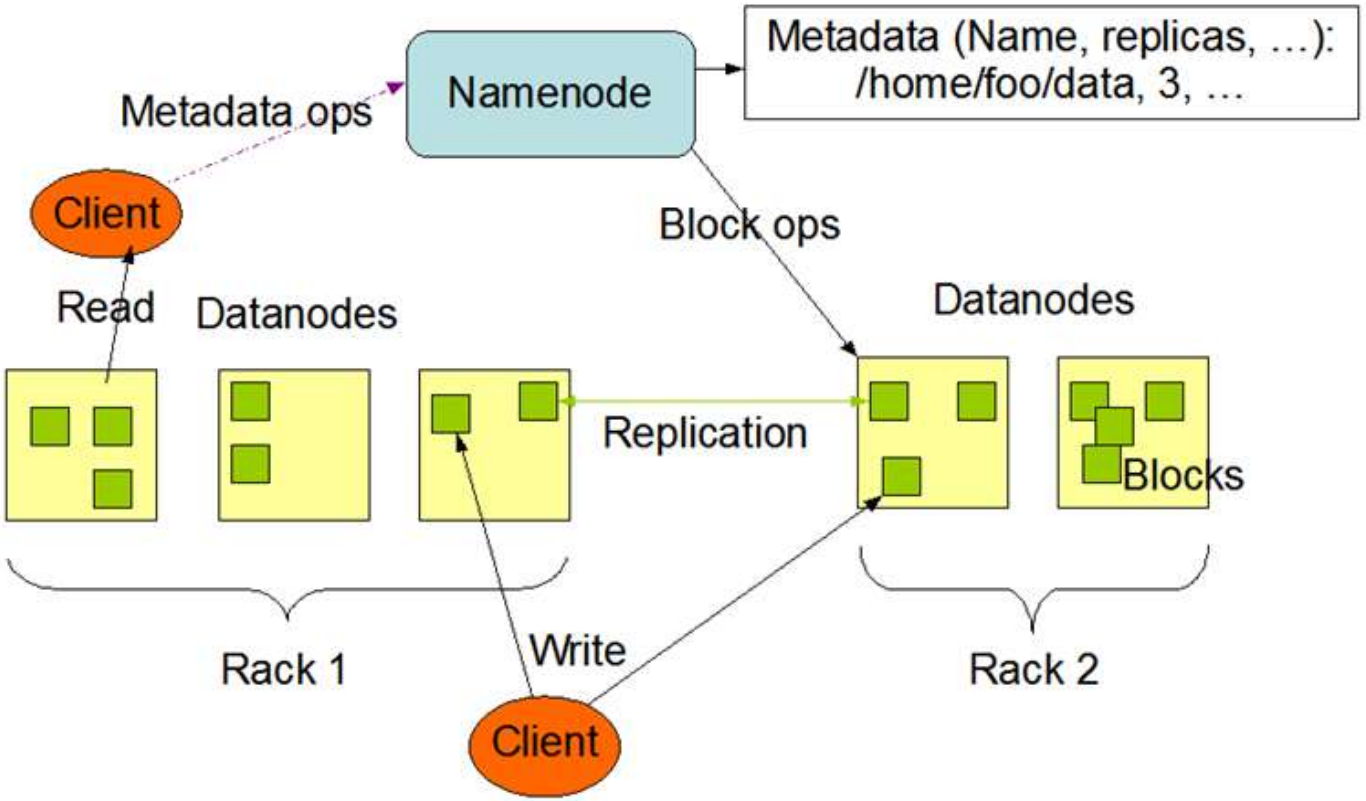
Anatomia unui cluster
[care sunt modulele care compun un cluster Hadoop]

- **NameNode**
  - Gestionează metadatele (informațiile despre sistem) pentru HDFS într-un fișier *fsimage*
- **Secondary NameNode**
  - Rulează funcții de întreținere pentru NameNode (NU este un backup pentru NameNode)
  - Actualizează fișierul *fsimage*
- **DataNode**
  - Deține blocurile de date din HDFS
- **JobTracker**
  - Administrează job-urile MapReduce determinând ordinea de execuție și alocarea task-urilor individuale.
- **TaskTracker**
  - Monitorizează (prin urmărirea performanței) task-urile individuale de tip Map si Reduce.

Ecosistemul Hadoop Ecosystem [alte unelte disponibile]

- De obicei, administrarea Hadoop este incomodă
- Uneltele permit programatorilor să beneficieze de întreaga putere a sistemului Hadoop.
- Hive
  - Procesare de SQL în Hadoop
- Pig
  - Procesare cu script-uri în
- Cascading
  - Model de procesarte cu Pipe și Filter
- HBase
  - Bază de date peste Hadoop
- Flume
  - Unealtă proiectată pentru mutarea datelor mari

Architectura HDFS



Architectura HDFS

- NameNode și DataNode:
  - Sunt aplicații software care rulează sub GNU/Linux.
  - HDFS este scris în Java. Orice sistem care are JVM poate rula NameNode și DataNode.
  - De obicei, un sistem rulează un singur NameNode.
- Aspecte principale:
  - Replicarea blocurilor
  - Persistența metainformațiilor sistemului de fișiere
    - NameNode folosește un fișier de log (i.e., EditLog) pentru a înregistra în mod persistent schimbările care au loc în sistemul de fișiere (ex., crearea unui nou fișier, schimbarea factorului de replicare, etc.)
    - Fișierul FsImage urmărește alocarile de blocuri pe noduri și proprietățile sistemului.

Architectura HDFS

- Aspecte principale:
  - **Protocoale de comunicație**: TCP/IP. Prin proiectare, un NameNode nu inițiază un RPC (Remote Procedure Call), el doar răspunde la RPC-urile trimise de clienți.
  - **Defectarea discurilor**: fiecare DataNode trimite periodic un mesaj *Heartbeat* către NameNode.
  - **Echilibrarea cluster-ului**: se realizează automat prim mutarea datelor de la DataNode la altul.
  - **Integritatea datelor**: un bloc de date dintr-un DataNode poate fi
  - a block of data from a DataNode may become stricat (ex., erori pe HD, erori în rețea, bug-uri software). HDFS implementează validarea cu ajutorul sumelor de control.
  - **Data race**: Politica implementată pentru fișiere este „write-once-read-many-times”.

Repere pentr instalarea și configurarea HDFS

- HDFS rulează în principal pe sisteme GNU/Linux.
- Rularea Hadoop in Windows:
  - Necesită Cygwin (i.e., emulator de Linux), care nu este recomandat în producție.
  - Pe o mașină virtuală cu Linux
- Cerințe SW: Java 1.6, Eclipse IDE și Cygwin (pentru Windows)
- Configurarea Hadoop:
  - fs.default.name; dfs.data.dir; dfs.name.dir; etc.
  - Trei moduri de lucry: Standalone, Pseudo-Distributed Mode, Fully-Distributed Mode
  - Pentru dezvoltarea aplicațiilor se poate folosi Plug-in pentru Eclipse:
    - Plug-in-ul este un fișier .jar care conține perspectiva *__Map/Reduce__*.

Utilizarea HDFS

- Start/stop HDFS
- Interacțiunea cu HDFS pentru încărcarea fisierelor și obținerea de informații se face prin două module:
  - **dfs** (i.e., FsShell): pentru lucrul cu fișiere
  - **dfsadmin** pentru interogarea și lucrul cu sistemul
- Comenzi pentru **dfs**:
  - Listarea fisierelor; crearea de directoare; încărcarea unui fișier; verificarea încărcării corecte a unui fișier; încărcarea mai multor fișiere în paralel; obținerea de informații de la HDFS; alte comenzi.
- Comenzi pentru **dfsadmin**:
  - Obținerea unui raport al stării; trecerea în safemode (numai operații read-only)

Utilizarea HDFS în mod programatic

```
4:  import org.apache.hadoop.conf.Configuration;
5:  import org.apache.hadoop.fs.FileSystem;
8:  import org.apache.hadoop.fs.Path;
10: public class HDFSHelloWorld {
15:   public static void main (String [] args) throws IOException {
17:     Configuration conf = new Configuration();
18:     FileSystem fs = FileSystem.get(conf);
20:     Path filenamePath = new Path(theFilename);
22:     try {
23:       if (fs.exists(filenamePath)) {
24:         // remove the file first
25:         fs.delete(filenamePath);
26:       }
27:
28:       FSDataOutputStream out = fs.create(filenamePath);
29:       out.writeUTF(message);
30:       out.close();
31:
32:       FSDataInputStream in = fs.open(filenamePath);
33:       String messageIn = in.readUTF();
34:       System.out.print(messageIn);
35:       in.close();
46:     } catch (IOException ioe) {
47:       System.err.println("IOException during operation: " + ioe.toString());
48:       System.exit(1);
49:     }
40:   }
41: }
```

Exemplu MapReduce

- https://hadoop.apache.org/docs/current/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html
- MapReduce constă din:
  - un singur master, și anume *ResourceManager*
  - Un worker de tip *NodeManager* pentru fiecare nod, și
  - *MRAppMaster* pentru toată aplicația (revizuiți ghidul arhitecturii YARN)
- Aplicațiile:
  - Specifică locațiile pentru input/output
  - Furnizează funcțiile *map* și *reduce* functions prin implementări ale interfețelor corespunzătoare si/sau ale claselor abstracte.
  - Și parametrii job-urilor sub forma unei configurații.

Exemplu MapReduce (cont.)

- Clentul Hadoop trimite job-ul (jar/executabil etc.) și configurația către ResourceManager care își asumă responsabilitatea distribuirii software-ului către workeri, programarea task-urilor și monitorizarea lor, furnizarea de informații referitoare la stare și disgnostic pentru job-ul client.
- **Input și Output**
  - Sistemul MapReduce lucrează exclusiv cu perechi <cheie, valoare>
  - (input) <k1, v1> -> map -> <k2, v2> -> combine -> <k2, v2> -> reduce -> <k3, v3> (output)

Exemplu WordCount

```
import org.apache.Hadoop.conf.Configuration, Job, Mapper, Reducer, etc…
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
```

Exemplu WordCount (cont.)

```
// the application
public class WordCount {
…
public static void main(String[] args) throws Exception {
}
}
```

Exemplu WordCount (cont.)

```
public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable>{
   private final static IntWritable one = new IntWritable(1);
   private Text word = new Text();// one line a time
   public void map(Object key, Text value, Context context
                ) throws IOException, InterruptedException {
     StringTokenizer itr = new StringTokenizer(value.toString());
     while (itr.hasMoreTokens()) {
       word.set(itr.nextToken());
       context.write(word, one);//Input (first map): <Hello, 1>; <World, 1>; <Bye, 1>; <World,1>
     }
   }
 }
```

Exemplu WordCount (cont.)

```
public static class IntSumReducer
      extends Reducer<Text,IntWritable,Text,IntWritable> {
   private IntWritable result = new IntWritable();
   public void reduce(Text key, Iterable<IntWritable> values,
                    Context context
                    ) throws IOException, InterruptedException {
     int sum = 0;
     for (IntWritable val : values) {
       sum += val.get();
     }
     result.set(sum);
     context.write(key, result); //Output (first map): <Hello, 1>; <World, 2>; <Bye, 1>
  }
 }
```